

TriVoKo

Multi-seller E-commerce Marketplace

Master Plan

Goals, outcomes, architecture, core flows, the four standout features, database, API, pages, testing, the 13-phase plan, risks and launch - explained with diagrams

Who uses TriVoKo and what each person can do



Real-life match: Customer = you on Flipkart · Seller = a shop selling on Flipkart · Admin = Flipkart's own staff.

Madhavan Suresh

Java Full Stack main portfolio project · React + Spring Boot + MySQL + Stripe
Version 1.0 · 1 October 2026 · planning document (before Phase 0)

Contents

1. What TriVoKo is and why we are building it	3
1.1 The goal: a job portfolio project	3
1.2 Decisions taken (planning session, 1 October 2026)	3
1.3 What "success" means.....	3
1.4 What we reuse from EventHub	4
2. Outcomes: what each person can do	5
2.1 A day on TriVoKo (the demo story).....	5
2.2 Outcomes per role	5
3. Architecture and technology	7
3.1 Why a modular monolith?	7
3.2 Technology list	8
3.3 Backend modules	8
3.4 Folder layout	9
4. Core flow: cart, checkout and the split order	10
4.1 Cart.....	10
4.2 Checkout - step by step.....	10
4.3 Money rules	11
4.4 Package and return states.....	11
5. The four standout features	13
5.1 Flash sale (standout #1)	13
5.2 Returns and refunds (standout #2).....	14
5.3 Price-drop alerts (standout #3).....	15
5.4 Smart search (standout #4).....	15
6. Database, API and pages.....	16
6.1 Main tables	16
6.2 API overview	16
6.3 Frontend pages	17
7. Security, testing and quality.....	18
7.1 Security checklist.....	18
7.2 Testing plan.....	18
8. The phase plan	20
8.1 Phases at a glance	20
8.2 Who types what	22
8.3 Must-have core and stretch.....	22
9. Scope, risks and launch	23
9.1 Not included (on purpose)	23

9.2 Risks and what we do about them..... 23

9.3 Launch and the portfolio outcome 24

9.4 Next steps after this plan 24

1. What TriVoKo is and why we are building it

TriVoKo is an online marketplace, like a small Flipkart. Many **sellers** open shops on TriVoKo and list their products. **Customers** search, put items from different sellers into one cart, pay once, and track every package. The **admin** team keeps the marketplace safe and running.

Real-life example: Think of Flipkart or Amazon. When you buy a phone from one shop and shoes from another shop in one order, you pay once, but two different shops pack and ship two different boxes. TriVoKo does exactly this.

1.1 The goal: a job portfolio project

TriVoKo is Madhavan's **main portfolio project**. It will be shown to recruiters and interviewers. That decides three things:

- It must be **live online**, so an interviewer can click a link and use it.
- It must look **polished** on laptop and phone.
- It needs **standout features** that are easy to demo and easy to explain in an interview.

1.2 Decisions taken (planning session, 1 October 2026)

Question	Decision
Why are we building it?	Job portfolio - live, polished, explainable
What kind of shop?	Marketplace - many sellers, one cart, orders split per seller
What is sold?	Everything - electronics, fashion, home, books (general, like Flipkart)
Standout features	Flash sale, Returns & refunds, Price-drop alerts, Smart search
Hosting	Decide at launch - the app is built portable (Docker + environment variables) so free or paid hosting both work
How we build	Claude builds each step and explains it; for the 3 core pieces (flash-sale stock, order split, return state machine) Madhavan types the key logic with guidance
Backend structure	Modular monolith - one Spring Boot app with walled modules; Redis only if the load test proves it is needed

1.3 What "success" means

TriVoKo is finished when all of these are true:

1. An interviewer opens the **live link**, logs in with a demo account and **buys from two sellers in one cart** with a Stripe test card.
2. They watch a **flash sale** start with a countdown, and the stock never goes below zero - proven by a **load test with 5,000 virtual users**.
3. They **return an item** and see the refund appear.
4. GitHub shows **CI green**, about **80% test coverage**, and a README with diagrams, GIFs and the load-test graph.

5. Madhavan can **explain every standout feature** in his own words: how it works, why it was built that way, and what would break without it.

1.4 What we reuse from EventHub

EventHub was the practice. Most building blocks of TriVoKo already exist there, so we copy the pattern and make it bigger.

EventHub feature	Becomes in TriVoKo
Seat hold for 10 minutes + @Version	Stock hold at checkout + flash-sale stock
100-thread "no overbooking" test	1,000-thread flash-sale test + k6 load test
Event approval state machine (DRAFT → PENDING → PUBLISHED)	Seller approval, product approval, package states, return states
Clubs with organizers	Sellers with their own shop dashboard
Stripe gateway switch + webhook + idempotency	Same, plus partial refunds for returns
Waitlist offer + notification bell + Mailpit emails	Price-drop alerts + order emails
JWT cookie + CSRF + roles	Same, with roles CUSTOMER / SELLER / ADMIN
Analytics with Recharts + @Cacheable	Seller sales charts + admin reports
Testcontainers, JaCoCo, Vitest, CI	Same, plus Playwright end-to-end tests
Docker image + Render/Vercel files	Same launch path, decided at Phase 12

Note: New things to learn in TriVoKo: product variants (size/colour), image upload to Cloudinary, splitting one order into packages, a money ledger, Stripe partial refunds, Hibernate Search (Lucene), k6 load testing and Playwright.

2. Outcomes: what each person can do

Who uses TriVoKo and what each person can do

 Customer (buyer) - Browse, smart search, filters - Product page: photos, sizes, reviews - One cart with items from many sellers - One payment (Stripe test mode) - Track each package separately - Return an item and get a refund - Wishlist + price-drop alerts - Coupons, PDF invoice, review after buying	 Seller (shop owner) - Apply to sell, admin approves the shop - Add products, sizes/colours, photos, stock "Orders to ship" list: Packed → Shipped - Approve or reject return requests - Join flash sales with a fixed stock - Earnings ledger (price minus 10% fee) - Sales charts for their shop only	 Admin (TriVoKo team) - Approve sellers and new products - See all users, orders and payments - Create coupons and schedule flash sales - Decide return disputes - Block bad sellers or users - Whole-site sales reports and charts - Audit log: who changed what
--	--	--

Real-life match: Customer = you on Flipkart · Seller = a shop selling on Flipkart · Admin = Flipkart's own staff.

Who uses TriVoKo and what each person can do

2.1 A day on TriVoKo (the demo story)

This story is also the script for the demo video and the interview walk-through.

1. **Ravi (customer)** searches "iphnoe 15". The typo is fixed and iPhone 15 cases show up with filters for brand and price.
2. He adds a phone case from **Seller A (Chennai Mobiles)** and running shoes from **Seller B (Kovai Sports)** to one cart.
3. At checkout he picks his saved address, applies coupon **WELCOME10** and pays **once** with the Stripe test card 4242 4242 4242 4242.
4. Order **TV-1001** is split into **two packages**. Chennai Mobiles sees only the phone case; Kovai Sports sees only the shoes.
5. Each seller marks their package **Packed → Shipped**. Ravi sees two tracking timelines and gets an email at each step.
6. The shoes do not fit. Ravi asks for a **return** with a reason and a photo. Kovai Sports approves, marks it picked up, and Ravi receives a **partial refund** of only the shoes (minus their share of the coupon).
7. **Kavya (customer)** had the same shoes in her wishlist. When Kovai Sports lowers the price, she gets a **price-drop alert**.
8. At 12:00 PM the **admin** starts a **flash sale**: 100 headphones at ₹999. Thousands click at the same moment; exactly 100 win, the rest see "Sold out" instantly.
9. The admin opens **reports**: sales per day, top sellers, return rate.

2.2 Outcomes per role

Customer

Can do	Shown on page
Browse categories, see deals and flash sales on the home page	Home

Can do	Shown on page
Search with typo tolerance and autocomplete; filter by category, brand, price, rating, in stock; sort	Search / listing
See photos, choose size/colour, read verified reviews, see "lowest price in 30 days"	Product page
Add to cart as a guest; the cart is kept after login	Cart
Save addresses, apply a coupon, pay once with Stripe	Checkout
Track every package of an order; download a PDF invoice	My orders / order detail
Cancel before shipping; return within 7 days after delivery	Order detail
Wishlist products and get price-drop alerts	Wishlist, bell, email
Review a product only after buying it	Order detail / product page

Seller

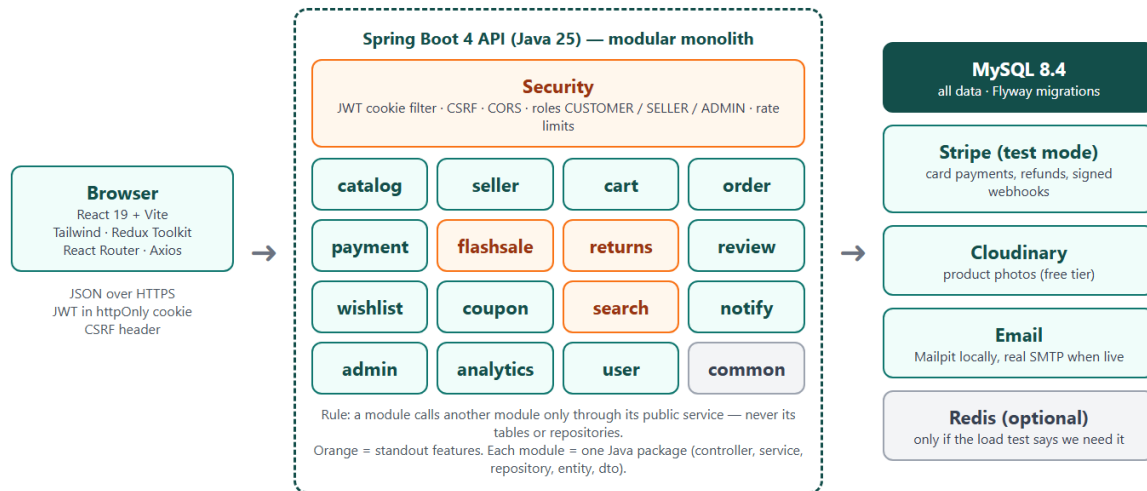
Can do	Shown on page
Apply to become a seller (shop name, city, GST number optional); wait for admin approval	Become a seller
Add and edit products with variants, photos (Cloudinary), price, MRP and stock	Seller → Products
See low-stock warnings	Seller dashboard
See only their own packages and move them Placed → Packed → Shipped (tracking number)	Seller → Orders
Approve or reject return requests (with a reason)	Seller → Returns
Join an admin flash sale with a sale price and a fixed quantity	Seller → Flash sales
See earnings: every sale adds 90% to the ledger, every refund takes it back	Seller → Earnings
See sales charts for their shop (7/30/90 days)	Seller → Analytics

Admin

Can do	Shown on page
Approve or reject seller applications and new products	Admin → Approvals
Search users, sellers and orders; block a seller or user	Admin → Users / Orders
Create coupons (flat or %, minimum order, expiry, one use per customer)	Admin → Coupons
Schedule flash sales and pick which seller offers join	Admin → Flash sales
Decide escalated return disputes	Admin → Returns
Site-wide reports: sales per day, top products, top sellers, return rate	Admin → Reports
Audit log: who changed what and when	Admin → Activity

3. Architecture and technology

TriVoKo architecture: one Spring Boot app with walled modules



TriVoKo architecture

3.1 Why a modular monolith?

Three ways to structure the backend were compared:

Option	Good	Bad	Decision
1. Modular monolith (one Spring Boot app, one package per module)	One app to host (free or cheap); easy to debug; clear module walls; can be split later	Cannot scale one module alone	Chosen
2. Microservices (product, order, payment services + gateway + Kafka)	Big buzzword	5+ apps to host - impossible on free hosting; weeks lost on plumbing instead of features	Rejected
3. Monolith + Redis from day one	Strong "high traffic" story	One more thing to host and learn before we know we need it	Only if the k6 load test shows MySQL is too slow

Real-life example: A modular monolith is like one big mall building: every shop (module) has its own walls and its own counter (service), and customers go through the counter - nobody walks into another shop's storeroom (tables). Later, a shop can move into its own building (microservice) without changing how customers talk to it.

Interview answer: "I chose a modular monolith because it is the right size for the traffic and the team. Each module only talks to others through its service interface, so splitting a module into a microservice later is a packaging change, not a rewrite. I measured the flash sale with k6 before deciding whether Redis was needed."

3.2 Technology list

Layer	Technology	Why
Frontend	React 19 + Vite, React Router, Redux Toolkit, Tailwind CSS v4, Axios	Same as EventHub - already known
Frontend extras	Recharts (charts), the useForm hook and UI kit copied from EventHub	Charts for seller and admin; less new code
Backend	Java 25, Spring Boot 4, Spring Web, Spring Data JPA, Spring Security, Validation	Same as EventHub
Database	MySQL 8.4 (Docker locally), Flyway migrations	Same as EventHub
Search	Hibernate Search 8 with the Lucene backend (index stored inside the app)	Typo tolerance + autocomplete without an extra server
Payments	Stripe Checkout (test mode), webhooks, partial refunds	Real payment flow - the carried-over EventHub to-do is done here
Images	Cloudinary free tier (signed upload from the browser)	Photos are not stored in MySQL or in the app
Email	Spring Mail; Mailpit locally; Brevo/SMTP free tier when live	Order and alert emails
PDF	OpenPDF	Invoices (like EventHub certificates)
Testing	JUnit 5, Mockito, Testcontainers, Vitest + React Testing Library, Playwright (end-to-end), k6 (load)	Proof for the README
Quality	JaCoCo coverage, oxlint/ESLint, GitHub Actions CI	Green badge on GitHub
Delivery	Docker image, environment variables, render.yaml / vercel.json or a VPS	Hosting decided at Phase 12
Optional	Redis	Only if the load test needs it

3.3 Backend modules

Every module is one Java package under `com.trivoko`, with its own `controller`, `service`, `repository`, `entity` and `dto` classes.

Module	Owns (tables)	Main job
<code>user</code>	users, user_roles, addresses	Register, login, profile, addresses
<code>seller</code>	sellers, seller_ledger	Seller applications, shop profile, earnings
<code>catalog</code>	categories, products, product_variants, product_images, price_history	Products and stock
<code>cart</code>	carts, cart_items	Cart for guests and users, merge on login
<code>order</code>	orders, packages, order_items	Checkout, order split, package states
<code>payment</code>	payments, refunds, processed_payment_events	Stripe Checkout, webhooks, refunds
<code>flashsale</code>	flash_sales, flash_sale_items, flash_sale_claims	Scheduled sales with atomic stock
<code>returns</code>	return_requests	Return state machine

Module	Owns (tables)	Main job
review	reviews	Verified-buyer reviews and rating average
wishlist	wishlist_items	Wishlist and price-drop alerts
coupon	coupons, coupon_redemptions	Coupon rules
search	(Lucene index)	Search, autocomplete, also-bought
notify	notifications	Bell + emails (after commit, with retry)
admin, analytics	audit_log	Approvals, reports, audit
common	-	Errors (ProblemDetail), money helpers, paging

Note: The module rule: a module may call another module's **service**, never its **repository** or entity tables. Modules announce changes with Spring events (for example **PackageDelivered**, **PriceChanged**) so that **notify**, **search** and **analytics** can react without being called directly. A small ArchUnit test checks this rule in CI.

3.4 Folder layout

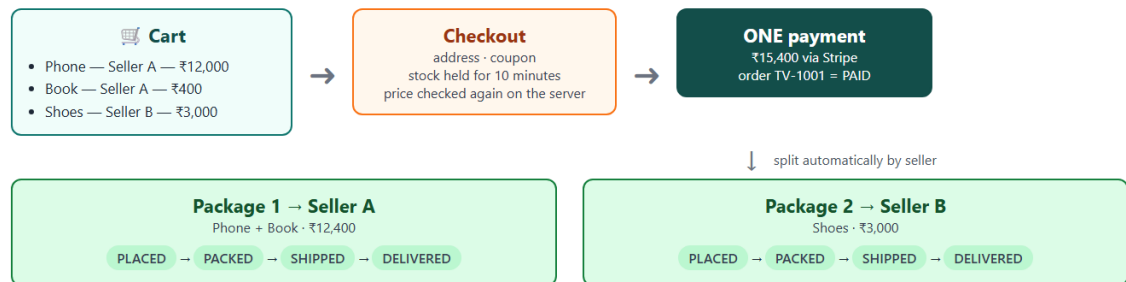
```

03_TriVoKo_Main_Project/
  00_Project_Documents/      master plan, phase plan, guides
  Phase_0_Setup/ ... Phase_12_Launch/  one checklist document per phase
  backend/                   Spring Boot app (package com.trivoko.*)
  frontend/                  React app
  docker-compose.yml         MySQL + Mailpit for development
  load-tests/                k6 scripts and results
  e2e/                       Playwright tests
  tools/                     document generators, demo scripts

```

4. Core flow: cart, checkout and the split order

Core feature: one cart, many sellers, one payment, split packages



The customer sees two tracking timelines inside one order. Seller A never sees Seller B's items or the customer's other purchases. Each package keeps its own money line: 90% to the seller's earnings, 10% TriVoKo fee.

One cart, many sellers, one payment, split packages

4.1 Cart

- A **guest** can add to cart. The cart lives in the browser (localStorage) with a guest token.
- After **login**, the guest cart is **merged** into the user's saved cart (same variant = quantities added, capped by stock).
- The cart shows items **grouped by seller**, so the customer already sees "this will come in 2 packages".
- Prices in the cart are only for display. **The server always recalculates prices at checkout** - a changed price is shown to the customer before paying.

4.2 Checkout - step by step

1. Customer chooses an address (or adds one) and optionally a coupon.
2. Server checks: every product still ACTIVE, every seller still APPROVED, coupon valid, quantities ≤ 10 per item.
3. Server **holds the stock** for every item in one transaction:

`UPDATE product_variants SET stock = stock - :qty WHERE id = :id AND stock >= :qty` If any line returns 0 rows, the whole checkout is rolled back and the customer is told which item ran out.

4. Server creates the **order** (status PENDING_PAYMENT, hold 10 minutes) and **one package per seller**. Order items **copy** the title, variant and price, so later product changes never change an old order.
5. Server creates a **Stripe Checkout session** for the full amount and the customer pays.
6. The Stripe **webhook** (or the "verify on return" fallback from EventHub) marks the order **PAID**; every package becomes **PLACED**; each seller gets an email; the coupon is marked used.

7. If nobody pays within 10 minutes, a job (every minute) marks the order **EXPIRED** and puts the stock back. A payment that arrives late re-takes the stock if still free, otherwise it is refunded automatically (same rule as EventHub).

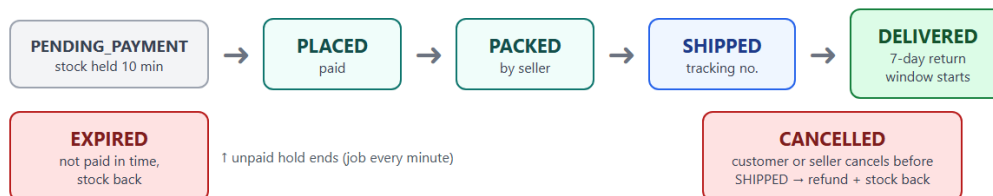
Real-life example: IRCTC Tatkal works like this: when you click "Book", the seat is kept for you while you pay. If the payment page times out, the seat goes back to the pool.

4.3 Money rules

- All money is stored as **DECIMAL(12, 2)** in rupees; Stripe receives **paise** (₹15,400.00 → 1540000).
- Coupon discount** is split across packages and items in proportion to their price, and stored on each order item. This makes partial refunds exact later.
- When a package is **DELIVERED**, the seller ledger gets **+90%** of that package's paid amount; TriVoKo keeps 10%. A refund adds a matching **minus** line. The seller's balance = sum of the ledger. (No real bank payouts - out of scope.)
- Shipping fee: free above ₹499 per package, otherwise ₹40 per package (simple fixed rule).

4.4 Package and return states

Package states (one package = one seller's part of an order)



Return states (standout feature #2)



Every move is checked by a state machine in Java (enum with allowed next states), exactly like EventHub's event approval — an illegal move (e.g. REFUNDED → REQUESTED) is a 409 error.

Package and return state machines

Move	Who	Rule
PENDING_PAYMENT → PLACED	system	Payment confirmed (webhook or verify)
PENDING_PAYMENT → EXPIRED	system job	10 minutes passed, not paid - stock back
PLACED → PACKED	seller	Own package only
PACKED → SHIPPED	seller	Tracking number required

Move	Who	Rule
SHIPPED → DELIVERED	seller (simulated courier) or demo button	Starts the 7-day return window, adds ledger line
PLACED / PACKED → CANCELLED	customer or seller	Before shipping only; refund that package; stock back

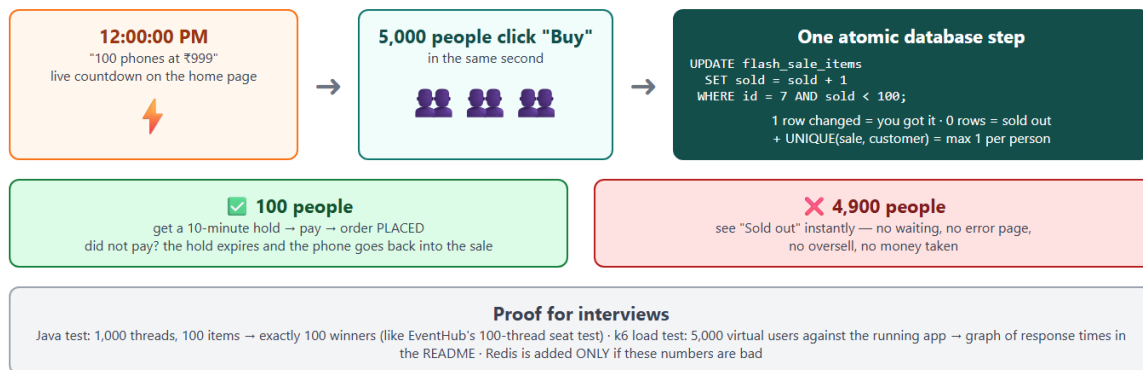
Note: Each state list is a Java `enum` with an `allowedNext()` method, exactly like EventHub's `EventStatus.nextAllowed`. Package rows have `@Version`, so if the customer cancels at the same moment the seller ships, only one wins and the other gets a clear 409 message.

5. The four standout features

These are the features that make TriVoKo different from a tutorial clone. Each one has a clear rule, a clear proof, and a clear interview story.

5.1 Flash sale (standout #1)

Standout feature #1: the flash sale that never oversells



The flash sale that never oversells

What the user sees: a "Flash sale starts in 00:04:59" banner on the home page. At 12:00 the button turns on, a live "73 left" counter drops, and when it reaches 0 the button changes to "Sold out".

How it works:

1. The admin creates a flash sale (start time, end time). Sellers offer variants with a sale price and a fixed quantity (for example 100). That quantity is **taken out of normal stock** when the sale is scheduled, so normal buyers and flash buyers never fight over the same units.
2. "Buy now" calls `POST /api/flash-sales/{id}/items/{itemId}/claim`.
3. The server runs **one atomic update**: `UPDATE flash_sale_items SET sold = sold + 1 WHERE id = ? AND sold < quantity AND NOW() BETWEEN start AND end`. One row changed = you got one. Zero rows = sold out (or not started / ended).
4. A row in `flash_sale_claims` with `UNIQUE(sale_item, customer)` stops anyone buying twice.
5. The winner gets a normal 10-minute checkout hold at the sale price. If they do not pay, the expiry job does `sold = sold - 1` and the unit is back in the sale.
6. When the sale ends, unsold units go back to normal stock.

Proof:

- Java test (like EventHub's `BookingConcurrencyTest`): 1,000 threads, 100 units → exactly 100 claims, `sold` = 100, never 101.
- **k6 load test**: 5,000 virtual users hit the running app at once. The README shows the result: requests per second, p95 response time, number of winners (must be exactly 100) and zero errors.
- **Only if** p95 is too slow (target: under 500 ms on a laptop), the counter moves to Redis (`DECR`), and we measure again and show both graphs.

Note: You type this part: the claim SQL, the unique check and the expiry give-back. Claude explains first and reviews after.

Interview story: "Instead of reading stock and then writing it - which lets two people both see 1 left - I let MySQL do check-and-decrement in a single statement, so the database itself guarantees no oversell. I proved it with a 1,000-thread test and a 5,000-user k6 run."

5.2 Returns and refunds (standout #2)

The return states are in the diagram in chapter 4.

Rules:

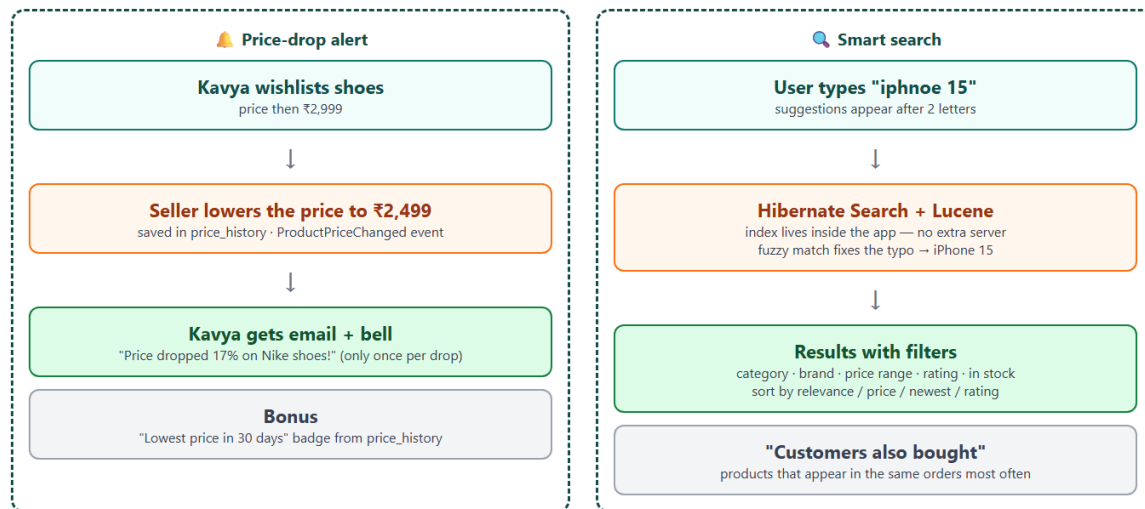
- A return can be requested only for a **DELIVERED** item, within **7 days**, once per order item, with a reason (list) and an optional photo (Cloudinary).
- The seller must answer within 2 days; if not, the request is **auto-approved** by a job (marketplace protects the buyer).
- If the seller rejects, the customer can **escalate once**; the admin makes the final decision.
- On **PICKED_UP**, the server calls **Stripe refund** for $\text{item price} \times \text{qty} - \text{item's coupon share}$ (a **partial refund** of the original payment). The refund is idempotent: a unique key per return request, so a double click or a retried webhook never refunds twice.
- Refund done → stock +qty, seller ledger minus line, customer email + bell.

Note: You type this part: the `ReturnStatus` enum with `allowedNext()`, and the service method that moves states. Claude writes the Stripe refund call and the tests around it.

Interview story: "Returns are a state machine. Every move is validated, every refund is idempotent, and money always matches: the ledger line, the Stripe refund and the stock change happen together."

5.3 Price-drop alerts (standout #3)

Standout features #3 and #4: price-drop alerts and smart search



Price-drop alerts and smart search

- Every price change writes a row to `price_history` and publishes a `PriceChanged` Spring event.
- A listener finds wishlist entries for that variant where the **new price is lower** than both the price when added and the last alerted price.
- One notification + email per customer per drop (stored `last_alert_price` stops spam if the seller changes the price 5 times in a minute - alerts are sent by a job every 10 minutes using the latest price).
- The product page shows "**Lowest price in 30 days**" using `price_history` - useful and honest.

5.4 Smart search (standout #4)

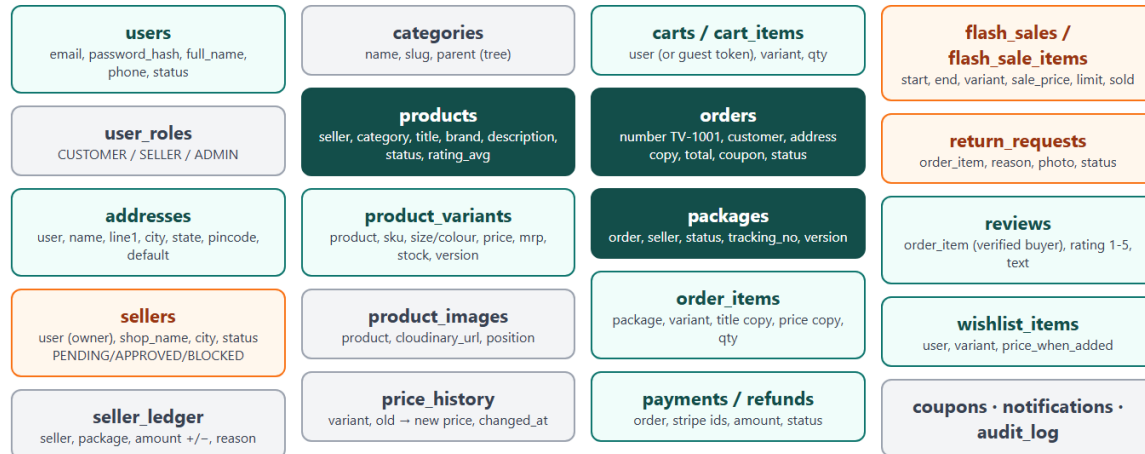
- **Hibernate Search** keeps a **Lucene index** of products inside the app (no Elasticsearch server to host). It updates automatically when a product changes.
- **Typo tolerance**: fuzzy matching ("iphnoe" → iPhone). **Autocomplete**: suggestions after 2 letters.
- **Filters and facets**: category, brand, price range, rating, in-stock - with counts ("Samsung (12)").
- **Sort**: relevance, price low/high, newest, rating.
- **Customers also bought**: products that appear together in paid orders most often (a SQL query, cached for 1 hour).

Note: If Hibernate Search causes trouble on free hosting (the index needs disk space), the fallback is MySQL FULLTEXT search without typo tolerance. This decision is checked in Phase 10.

6. Database, API and pages

6.1 Main tables

Main tables and how they are linked



Read it like this: a **seller** owns many **products**; a product has **variants** (size/colour, each with its own price and stock); an **order** is split into **packages** (one per seller); each package has **order_items**. Order items COPY the title and price, so later price changes never change old orders.

Main tables and how they are linked

Important database rules:

- **Order items copy** the product title, variant text and price. An old invoice never changes.
- **Stock lives on the variant** (size M of a T-shirt has its own stock), not on the product.
- Rows that many people change at once (**product_variants**, **packages**, **flash_sale_items**) have a **version** column or use a single atomic **UPDATE ... WHERE**.
- Status columns are **VARCHAR + CHECK** (not MySQL **ENUM**) - the EventHub rule.
- Every table change is a **Flyway migration** (V1, V2, ...). Nothing is changed by hand.

6.2 API overview

Group	Examples	Who
Auth	POST /api/auth/register, /login, /logout, GET /api/auth/me	anyone
Catalogue	GET /api/products?q=&category=&brand=&min=&max=&sort=, GET /api/products/{slug}, GET /api/categories	anyone
Search	GET /api/search?q=, GET /api/search/suggest?q=	anyone
Cart	GET/PUT/DELETE /api/cart/items, POST /api/cart/merge	customer
Checkout	POST /api/checkout → order + Stripe URL, POST /api/payments/{session}/verify	customer
Orders	GET /api/orders, GET /api/orders/{number}, POST /api/packages/{id}/cancel, GET /api/orders/{number}/invoice.pdf	customer (own)

Group	Examples	Who
Returns	POST /api/order-items/{id}/returns, POST /api/returns/{id}/escalate	customer (own)
Wishlist & reviews	POST/DELETE /api/wishlist/{variantId}, PUT /api/order-items/{id}/review	customer
Flash sales	GET /api/flash-sales/live, POST /api/flash-sales/{id}/items/{itemId}/claim	anyone / customer
Seller	/api/seller/products, /api/seller/packages/{id}/pack, /ship, /deliver, /api/seller/returns/{id}/approve, /reject, /picked-up, /api/seller/earnings, /api/seller/analytics	approved seller (own data only)
Admin	/api/admin/sellers/{id}/approve, /api/admin/products/pending, /api/admin/coupons, /api/admin/flash-sales, /api/admin/returns/{id}/decide, /api/admin/reports	admin
Payments	POST /api/payments/stripe/webhook (signed)	Stripe only

All errors use the EventHub **ProblemDetail** format (400 / 401 / 403 / 404 / 409 / 429) with a clear message for each field. Swagger shows the full list (admin only when live).

6.3 Frontend pages

Area	Pages
Shop (public)	Home (deals, flash-sale banner, categories), Search/listing with filters, Product page, Seller shop page, Cart, Login, Register, Become a seller
Customer	Checkout, Payment result, My orders, Order detail (tracking timelines, invoice, cancel, return, review), Wishlist, Addresses, Notifications
Seller	Dashboard (today's orders, low stock, earnings), Products list + product editor (variants, photos), Orders to ship, Returns, Flash-sale offers, Earnings ledger, Analytics
Admin	Overview, Seller approvals, Product approvals, Users, Orders, Coupons, Flash sales, Return disputes, Reports, Activity log
Other	404 page, Style guide (dev only), dark mode on every page, mobile layout on every page

Note: Design: a fresh TriVoKo look (teal + saffron colours, new logo), built on the EventHub UI kit (Button, Card, Badge, Modal, FormField, Toaster). Sketches of the 6 key pages are made in Phase 0 and approved before coding the frontend.

7. Security, testing and quality

7.1 Security checklist

Risk	Protection
Stolen login	JWT in an httpOnly, Secure, SameSite cookie; BCrypt passwords; 5 failed logins → 15-minute lock (EventHub LoginAttemptService)
Forged requests from other sites	CSRF double-submit token (EventHub pattern)
Seller A reading Seller B's orders	Every seller query is filtered by the logged-in seller's id; @PreAuthorize ownership checks; tests that try to read another seller's package must get 403/404
Customer changing prices in the browser	Server recalculates every price, discount and shipping fee at checkout
Fake payment confirmation	Stripe webhook signature check ; idempotent processing (EventHub processed_payment_events)
Double refund	One refund key per return request; Stripe idempotency key
Bot spamming the flash sale	Rate limit per user/IP on claim (429 + Retry-After); one claim per customer
Bad file uploads	Signed Cloudinary uploads only, images only, max 5 MB, max 8 photos
Secrets in Git	.env git-ignored; secret scan of the history before launch
Admin tools exposed	Swagger and Actuator for ADMIN only in production

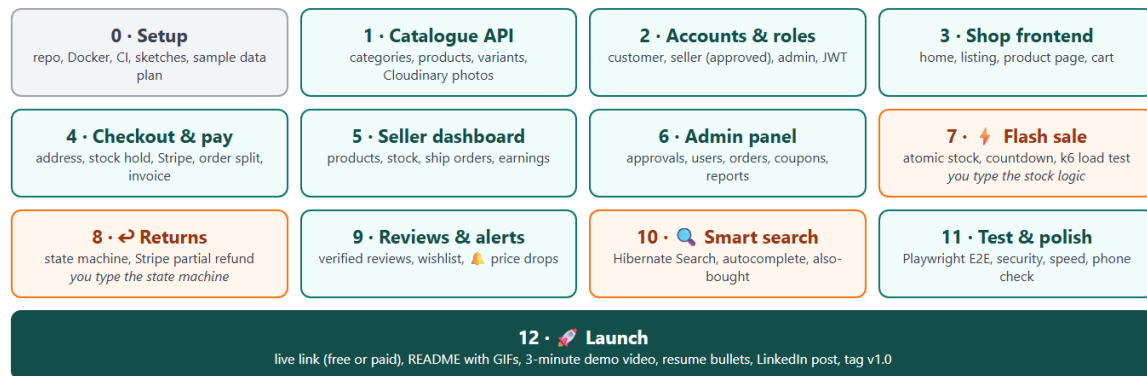
7.2 Testing plan

Level	Tool	What is tested	Target
Unit	JUnit 5 + Mockito	Money maths (coupon split, refund amount, ledger), state machines, rules	Every rule
Integration	@SpringBootTest + MockMvc + Testcontainers MySQL	Every endpoint, security per role, flows (checkout → pay → split → ship → return → refund)	Service coverage ~80% (JaCoCo)
Concurrency	Plain Java threads	Flash sale 1,000 threads / 100 units; checkout stock hold 100 threads / 10 units; cancel-vs-ship race	Never oversell, always one winner
Frontend	Vitest + React Testing Library	Cart maths, forms, countdown, key components	Main components
End-to-end	Playwright	The demo story in a real browser: buy from 2 sellers, seller ships, customer returns	Runs in CI
Load	k6	Flash sale with 5,000 virtual users; product listing under load	Exactly 100 winners, p95 < 500 ms locally
Manual	Phone + laptop, light + dark	Every page	No layout bugs

Note: Every phase ends with: tests pass locally, **CI green on GitHub**, commit pushed, and the "done when" checks of that phase shown working in the browser.

8. The phase plan

The 13 phases (each phase ends with a working, tested, pushed app)



Must-have core = phases 0-8 (a complete marketplace with the two strongest standouts). Phases 9-10 make it shine. If time runs short, 9 and 10 can be shortened without breaking anything.

The 13 phases

Each phase gets its own folder (**Phase_N_Name**) with a checklist document - like EventHub. The table below is the summary; the detailed step checklists are written next, after this master plan is approved.

8.1 Phases at a glance

Phase	Goal	Main work	Done when
0 Setup	Project ready to build	Git repo trivoko on GitHub, Spring Boot + React skeletons, docker-compose (MySQL, Mailpit), CI, .env , logo and colours, sketches of 6 key pages, sample-data plan (8 sellers, 120 products, 10 categories)	React shows "Backend: UP"; CI green; sketches approved
1 Catalogue API	Products exist	Tables categories / products / variants / images / price_history; seed data; listing with filters, paging and sorting; product detail; Cloudinary signed upload	GET /api/products?category=phones&sort=price returns correct pages; Postman checks pass
2 Accounts & roles	Safe login for 3 roles	Register/login/logout/me (EventHub auth), addresses, "Become a seller" + admin approval, ownership checks, demo accounts	Seller cannot touch another seller's product (test proves 403); unapproved seller cannot sell
3 Shop frontend	Customer can shop (no payment yet)	Home, listing with filters in the URL, product page with variant picker and gallery, cart grouped by seller, guest cart + merge on login, dark mode, mobile	Guest adds 3 items from 2 sellers, logs in, cart is kept

Phase	Goal	Main work	Done when
4 Checkout & payments	Real orders	Stock hold, order + packages split, coupons, Stripe Checkout + webhook + verify fallback, expiry job, late-payment rule, emails, PDF invoice, real Stripe 4242 test with Stripe CLI (EventHub to-do)	Pay once for 2 sellers → 2 packages PLACED; 100-thread stock test passes
5 Seller dashboard	Sellers run their shop	Product editor with variants + photos, stock warnings, orders to ship (Packed → Shipped → Delivered), earnings ledger, analytics charts	Seller ships a package and the customer sees the timeline update
6 Admin panel	Admin controls the marketplace	Seller + product approvals, users/orders search, block, coupons, reports, audit log	Admin blocks a seller → their products disappear from the shop
7 Flash sale	Standout #1	Schedule sale, reserve stock, atomic claim, one per customer, countdown + live counter, rate limit, give-back job, 1,000-thread test, k6 5,000 users , Redis only if needed	k6 report: exactly 100 winners, 0 oversell, p95 recorded in README
8 Returns & refunds	Standout #2	ReturnStatus state machine, photo upload, seller approve/reject, auto-approve job, escalate + admin decision, Stripe partial refund, stock + ledger update	Return one item of a 2-item order → exact partial refund in Stripe test dashboard
9 Reviews, wishlist, alerts	Standout #3	Verified-buyer reviews + rating average, wishlist, price_history, price-drop job + email + bell, "lowest in 30 days"	Lower a price → wishlisted customer gets one alert (not five)
10 Smart search	Standout #4	Hibernate Search + Lucene, fuzzy match, autocomplete, facets with counts, "customers also bought"	"iphnoe" finds iPhone; suggestions appear while typing
11 Testing & polish	Portfolio quality	Playwright demo-story test in CI, coverage ~80%, ArchUnit module rule, security review, accessibility + phone check, speed (lazy pages, image sizes), README draft	CI green with E2E; Lighthouse ≥ 90 on home page

Phase	Goal	Main work	Done when
12 Launch	Live and shown	Choose hosting (free or paid), deploy backend + DB + frontend, real email, demo accounts, README with GIFs + architecture + k6 graph, 3-minute demo video, resume bullets, LinkedIn post, tag v1.0	A friend opens the link on a phone, buys, returns, and it all works

8.2 Who types what

Part	Built by
Setup, configuration, copies of EventHub patterns (auth, errors, UI kit, Stripe gateway, emails)	Claude builds and explains line by line
Catalogue, cart, seller and admin pages, reports	Claude builds and explains; Madhavan reviews and asks for changes
Order split (grouping cart lines into packages, coupon share per item)	Madhavan types the key method with guidance, Claude reviews
Flash-sale claim (atomic SQL, one-per-customer, give-back)	Madhavan types with guidance, Claude reviews
Return state machine (<code>ReturnStatus.allowedNext</code> , state-move service)	Madhavan types with guidance, Claude reviews
Tests for the three parts above	Written together - Madhavan writes at least one test for each

8.3 Must-have core and stretch

- **Must-have core = Phases 0-8.** This is already a complete marketplace with the two strongest standouts (flash sale + returns).
- **Shine = Phases 9-10.** If time is short, 9 and 10 can be made smaller (for example search without facets) without breaking anything else.
- **Always = Phases 11-12.** A portfolio project that is not tested, not live and not explained is not finished.

9. Scope, risks and launch

9.1 Not included (on purpose)

Left out	Why	What we do instead
Real courier API (Delhivery, Shiprocket)	Needs a business account	Seller clicks Shipped/Delivered; tracking timeline is simulated
Real bank payouts to sellers	Needs Stripe Connect + KYC	Earnings ledger shows what each seller would receive
Cash on delivery, UPI, EMI	More payment flows, little extra learning	Stripe test card only
Seller-created coupons	Doubles the coupon rules	Admin coupons only
Mobile app, multiple languages	Big extra work	Mobile-friendly website
Full GST invoicing	Tax rules are complex	Simple invoice PDF with price, discount, shipping, total
Chat between buyer and seller	Real-time features are a separate project	Return reason + seller reply text

9.2 Risks and what we do about them

Risk	Plan
Project is too big and never finishes	Core = Phases 0-8 first; every phase ends with a working, pushed app; 9-10 can shrink
Free hosting is slow to wake up	Hosting chosen at Phase 12; uptime pinger for free hosting, or a paid plan only during job-hunting months
Hibernate Search needs disk on the host	Index can be rebuilt on start-up (120-500 products takes seconds); fallback = MySQL FULLTEXT
Stripe test webhooks are hard locally	EventHub "verify on return" fallback; Stripe CLI in Phase 4
Cloudinary or email free limits	Small sample images; emails only for important steps; Mailpit locally
Madhavan cannot explain a part in an interview	The 3 core parts are typed by him; every phase ends with a short "explain it back" check; viva Q&A in the final guide
Spring Boot 4 / Java 25 library gaps	Same versions as EventHub, which already work; check each new library (Hibernate Search, Playwright, k6) in a small spike first

9.3 Launch and the portfolio outcome

The finished outcome: what you show a recruiter



Live link
trivoko.... opens in 1-2 s, demo logins for each role on the login page



GitHub
CI green · ~80% test coverage · README with architecture diagram, GIFs and load-test graph



3-minute demo video
buy from 2 sellers → flash sale → return & refund



Interview stories
"How did you stop overselling?"
"How do refunds work?"
"Why a modular monolith?"

Resume line
Built **TriVoKo**, a multi-seller e-commerce marketplace (Spring Boot 4, React 19, MySQL, Stripe) with split-order fulfilment, a flash-sale engine load-tested at 5,000 concurrent users with zero oversell, a return-and-refund workflow, price-drop alerts and typo-tolerant search; CI with ~80% coverage, deployed with Docker.

What you show a recruiter

Hosting choices at Phase 12 (the code is the same for all of them):

Choice	Cost	First load	Notes
Free: Render + Aiven MySQL + Vercel	₹0	30-60 s when asleep	Uptime pinger keeps it warm during job hunting
Small paid: Railway or a VPS	about ₹400-600 per month	1-2 s	Can be paid only for the months of applying

Launch package:

- Live link with **demo logins** for customer, seller and admin on the login page.
- GitHub README: what it is, GIFs of the demo story, architecture diagram, ER diagram, k6 graph, how to run locally, test coverage badge.
- **3-minute demo video** (English; Tamil version optional, like EventHub).
- Resume bullet and LinkedIn post.
- Final project guide + viva Q&A document (like the EventHub 105-page guide).
- Git tag **v1.0**.

9.4 Next steps after this plan

1. Madhavan reviews this master plan and asks for changes.
2. Claude writes the **TriVoKo Phase Plan** (step-by-step checklist for every phase, with "done when" checks) and the Phase folders.
3. Phase 0 starts: GitHub repo **trivoko**, skeleton apps, Docker, CI, logo and sketches.